

PROJECT BLUEPRINT: Maitri AI Exercise Platform

1. Executive Project Summary

Maitri is a high-performance, real-time AI rehabilitation and physical therapy platform. It leverages computer vision to provide instantaneous feedback on exercise form, bridging the gap between clinical guidance and home-based recovery. By analyzing skeletal posture at 30 FPS, Maitri identifies biomechanical flaws (e.g., knee valgus during squats) and provides corrective audio-visual cues to ensure safety and maximize therapeutic efficacy.

2. System & Feature Architecture

Technical Stack

- **Backend (Core Engine):** Python 3.11.9, FastAPI (Asynchronous REST & WebSockets), MediaPipe (Pose Landmark Extraction), OpenCV, NumPy.
- **Frontend (Application):** Vite + React + TypeScript, Tailwind CSS, Lucide React (Iconography), Recharts (Biometric Data Visualization).
- **Intelligence:** Google Gemini (Automated Rehabilitation Plan Generation).
- **Communication:** Bidirectional WebSockets for telemetry; HTTP REST for configuration and metadata.

System Architecture

Maitri utilizes a **Client-Server Separation** model optimized for high-frequency data processing:

- **Frontend Responsibilities:** Local webcam capture (`getUserMedia`), skeletal HUD rendering (HTML5 Canvas), biometric visualization, and client-side Text-to-Speech (Web Speech API).
- **Backend Responsibilities:** MediaPipe pose inference, exercise-specific rule evaluation, and AI-driven clinical reasoning.

Core Feature Map

Feature	Implementation	File/Module
Pose Estimation	MediaPipe skeletal tracking (33 landmarks)	core/pose_engine.py
Exercise Logic	Modular plugin-based rule engines	exercises/<name>/analyser.py
Telemetry Stream	30 FPS JPEG-to-JSON WebSocket pipeline	backend/server.py, backend/pipeline.py
Rehab AI	LLM-based injury-to-exercise mapping	backend/rehab.py
Audio Feedback	Non-blocking multi-layered TTS system	core/audio.py (Server) / Web Speech API (Client)
Visual HUD	Responsive glassmorphic overlay	frontend/src/components/HUD.tsx

3. Component Breakdown

Core Infrastructure (/core)

- **pose_engine.py**: A wrapper for MediaPipe that normalizes coordinates into a structured `PoseLandmarks` dataclass.
- **geometry.py**: Pure functional math helpers for calculating 3D angles and vector distances.
- **base_analyser.py**: Defines the `BaseAnalyser` and `BaseResult` ABCs, ensuring a strict contract for all exercise plugins.

Exercise Plugins (/exercises)

- **squats/analyser.py**: Implements specialized logic for rep counting, depth detection, and common form errors like "knee cave" or "forward lean".
- **__init__.py**: Contains the `EXERCISE_REGISTRY`, allowing new exercises to be "hot-swapped" into the system with zero changes to the core server.

Backend API (/backend)

- **server.py**: The FastAPI entry point. Manages WebSocket connections and routes frames to the pipeline.
- **pipeline.py**: Orchestrates the per-frame flow: Decoding → Landmark Extraction → Rule Evaluation → Serialization.
- **rehab.py**: Interface for Gemini 1.5 Flash to process natural language injury descriptions into structured exercise recommendations.

Frontend Application (/frontend)

- **src/hooks/useWebSocket.ts**: Handles the base64-encoding of video frames and synchronizes incoming telemetry with the React state.
- **src/components/SkeletonOverlay.tsx**: Maps 3D backend landmarks onto the local 2D video coordinate space using Canvas API.

4. Key Engineering Decisions

Latency-First Processing

To achieve 30 FPS, the backend uses `asyncio.get_event_loop().run_in_executor()` to offload heavy MediaPipe CPU tasks. This prevents the WebSocket handler from blocking while processing a frame, allowing the system to handle jitter and packet loss gracefully.

Client-Side Video / Server-Side Logic

The frontend captures video locally to provide zero-latency visual feedback (the user sees themselves instantly). Only the "intelligence" (pose analysis) is offloaded to the server. This "thin-logic" server approach minimizes the data bandwidth compared to streaming high-res video back and forth.

Base64 JPEG Stream

While binary protocols like Protobuf were considered, **Base64-encoded JPEG over WebSockets** was chosen for its ease of debugging and native compatibility with browser-side **Canvas** and **Image** objects without specialized decoders.

5. Data Flow & Logic

1. **Capture:** Frontend draws the current webcam frame to an offscreen canvas and exports it as a low-quality JPEG (0.6 quality) to minimize payload size.
 2. **Ingest:** The frame is sent via WebSocket to the `/ws` endpoint.
 3. **Analyze:**
 - `pose_engine` extracts (x, y, z) landmarks.
 - `analyser` (e.g., Squats) calculates the `knee_angle`.
 - If `knee_angle` $< 90^\circ$, the phase transitions from `descending` to `ascending`.
 4. **Respond:** Backend sends a JSON payload containing `rep_count`, `has_issue: true/false`, and a `feedback: "Push knees out"` string.
 5. **Visualize:** Frontend triggers a "pulse" animation on the rep counter and speaks the feedback via TTS if `rep_just_completed` or `is_urgent` is flagged.
-

6. Current Status & Future Roadmap

Current Status

- **✓ Core Engine:** MediaPipe pose extraction and 3D geometry engine functional.
- **✓ Squat Analyser:** Robust rep counting and error detection (knee cave, depth).
- **✓ Communication:** 30 FPS WebSocket pipeline stable.
- **✓ AI Rehab:** Gemini integration for injury-based planning is live.
- **✓ Frontend:** High-contrast dark mode UI with real-time sparklines implemented.

Future Roadmap

1. **Exercise Expansion:** Implement Lunges, Deadlifts, and Shoulder Mobility analysers.
2. **Persistence Layer:** Integrate a database (PostgreSQL/Supabase) to track user progress and "Heatmaps" of form improvement over time.
3. **Mobile Optimization:** Progressive Web App (PWA) support with client-side MediaPipe (Wasm) for offline-first analysis.
4. **Multi-Person Tracking:** Add logic to isolate the primary user in crowded environments.